

Scheda — FI2 LAB04b: Frazione «double face»

Materiale: LAB-04-Insiemi di frazioni.pdf (sl. 22-40), Lab04b-FrazioniDoubleFace-Startkit.zip (FrazioneTest, MyMain, MyMath) · **Modalità:** guidata · **Tempo del docente:** 30 minuti [sl. 40] **Teoria che entra qui:** 07 (in corso) — array di riferimenti «inizialmente tutti null» sl. 14-15, length sl. 19 · 05 □ — overloading sl. 57-58 (stesso nome, liste di argomenti diverse; il tipo di ritorno non basta) · 04b (in corso) — classe con soli membri statici sl. 9-12.

Setup

- dopo `git pull` in `~/cockpit`: *Existing Projects* → *Select root directory* → `esame_FI2/da_importare/LAB04b_FrazioniDoubleFace-011026` → **Copy projects into workspace** → **Finish**. Già rinominato, compliance 21, configurazione `FrazioneTest` con `-ea`;
- `frazione/Frazione.java` è **la tua** del LAB04a, già copiata; `FrazioneTest` e `MyMain` sono dello startkit (sl. 38);
- X rosse attese in `FrazioneTest` e `MyMain`: *The method `sum(Frazione)` in the type `Frazione` is not applicable for the arguments `(Frazione[], Frazione[])`* — esiste solo il `sum` d'istanza; e `convertToString` non esiste.

Ordine di lavoro

#	In Frazione	Firma (sl. 26, 40)	Test	Costrutto → slide
1	ex <code>FrazLib</code>	<code>public static Frazione sum(Frazione[] tutte), idem mul</code>	nessuno (ma sl. 26 li vuole)	metodi statici nella classe-tipo — sl. 23-24
2	stampa	<code>public static String convertToString(Frazione[] tutte)</code>	<code>MyMain</code> a video	è sulla sl. 28: copiala e capiscila riga per riga
3	dimensione logica	<code>public static int size(Frazione[] tutte)</code>	indiretto	primo <code>null</code> = fine — sl. 32

#	In Frazione	Firma (sl. 26, 40)	Test	Costrutto → slide
4	somma a coppie	public static Frazione[] sum(Frazione[] setA, Frazione[] setB)	4 assert su somma	traccia in 3 passi sl. 28; overloading — 05 sl. 57
5	prodotto a coppie	public static Frazione[] mul(Frazione[] setA, Frazione[] setB)	4 assert su prodotto	«idem» — sl. 28

Casi limite da pensare

- Ora Frazione ha **tre** metodi sum. Come capisce il compilatore quale chiami in `setA[k].sum(setB[k])` e quale in `Frazione.sum(a, b)`? (05 sl. 57.)
- Gli array del test hanno length 10 ma 4 frazioni. Che cosa c'è in `collezioneA[4]`, e che cosa succede se il tuo ciclo ci arriva? (07 sl. 14; LAB sl. 36.)
- Il `sum(Frazione[])` che porti da `FrazLib` usa il *for each*: su un array come questi, regge? (△ già segnato nel confronto LAB04a, «cella null».)
- Se le due dimensioni logiche differiscono, che cosa restituisci, e perché la sl. 30 lo chiama «il solo modo che abbiamo per lanciare un allarme (per ora..)»?
- Quanto deve essere lungo l'array risultato: 10 o 4? (sl. 30, nota in basso.)
- Il quarto assert di somma vuole 0/6. Con la tua `sum` d'istanza, quanto fa $1/6 + (-1/6)$ — **denominatore compreso**? Quale dei tuoi metodi di somma dà il denominatore che il test si aspetta? (△ confronto LAB03, riga 1: `sum` contro `sumWithMcm`; il test è il contratto.)
- △ Pattern 2: i test toccano solo i metodi a due array; i passi 1–3 sono finiti quando li hai verificati tu (con `MyMain`).

Test rosso?

Non compila: firma diversa da quella del test (`static`, tipo di ritorno `Frazione[]`) · `AssertionError` alla riga `N` = failure: guarda quale cella e quale campo · `NullPointerException` = error: il ciclo è entrato in una cella null → prontuario §1, oppure scrivilo in chat. Silenzio = verde **solo con -ea**.

Dopo i test verdi

- copia il progetto in `corsi/FI2/esame_FI2/svolti/LAB04b_FrazioniDoubleFace/`;
- confronto con la soluzione del docente → `confronto_LAB04b.md`;

- poi LAB04c (sl. 41-42): «l'uso diretto di array non è un'idea meravigliosa» → ADT `FractionCollection`.